

Práctica 1

Parte 3

En la tercera parte de la práctica vamos a incorporar un elemento externo al microcontrolador: un botón pulsador. A pesar de su simplicidad, el botón introduce la necesidad de que nuestro código se vea afectado por el estado de una señal binaria externa. Para esto, utilizaremos un componente hardware presente en cualquier microcontrolador: el **GPIO** (General Purpose Input Output). Cada GPIO es un circuito que controla un pin del chip, y que puede configurarse como entrada (como es nuestro caso) o como salida. Configurado como entrada puede detectar niveles de voltaje (alto o bajo); configurado como salida puede generar niveles de voltaje, lo cual puede servir para controlar o comunicarse con dispositivos externos. En resumen, proporciona una interfaz flexible para la interacción con el entorno externo del microcontrolador.

En este caso, el pin del chip donde conectemos el botón actúa como entrada, y como tal es un recurso externo al que dedicar recursos para conocer su estado. Vamos a explorar dos estrategias para hacerlo: **polling** e **interrupciones**.

1. Polling:

Este enfoque implica revisar repetidamente el estado de un recurso como parte de nuestro de programa. Este método es directo y fácil de entender, ideal para situaciones donde la eficiencia no es crítica y la simplicidad es clave.

2. Interrupciones:

Permiten al microcontrolador "escuchar" cambios en el estado del recurso sin la necesidad constante de revisar su estado (es decir, sin dedicar recursos de cómputo a esta tarea). Este método es particularmente valioso en sistemas donde la eficiencia y la respuesta rápida son esenciales.

Ambos enfoques tienen sus ventajas y desventajas (y ambos son ampliamente), y seleccionar el método adecuado depende de los requerimientos de la aplicación. Vamos a experimentar con ambos para adquirir una visión más completa de cómo gestionar la entrada de hardware en entornos empotrados. Sigue los siguientes pasos:

1. Crea el proyecto STR2024_P1_3.
2. En esta parte de la práctica vamos a trabajar con diferentes *environments de Platformio*. Estos son útiles cuando tu proyecto tiene diferentes configuraciones (como las típicas debug y release de muchos entornos de programación). Modifica el fichero platformio.ini para que contenga lo siguiente:

```
[env:polling]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
upload_port = <tu_puerto>
monitor_port = <tu_puerto>
board_build.cmake_extra_args = -D MODE=polling

[env:interrupt]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
upload_port = <tu_puerto>
monitor_port = <tu_puerto>
board_build.cmake_extra_args = -D MODE=interrupt
```

En este caso, vamos a crear dos configuraciones del proyecto: “polling” e “interrupt”, cuyos ficheros fuente estarán en dos carpetas separadas, del mismo nombre y colgando del directorio src. Puesto que ESP-IDF utiliza CMake para compilar, las configuraciones pasan una variable a CMake para que incluya diferentes directorios de fuentes.

Para saber más de las configuraciones de Platformio, échale un vistazo a la documentación, es muy útil para gestionar proyectos más complejos:

<https://docs.platformio.org/en/latest/projectconf/sections/env/index.html>

Observa que hay dos CMakeLists.txt en el proyecto: STR2024_P1_3/CMakeLists.txt, y otro en STR2024_P1_3/src/CMakeLists.txt. Abre el primero de ellos y modifícalo para que tenga estas líneas:

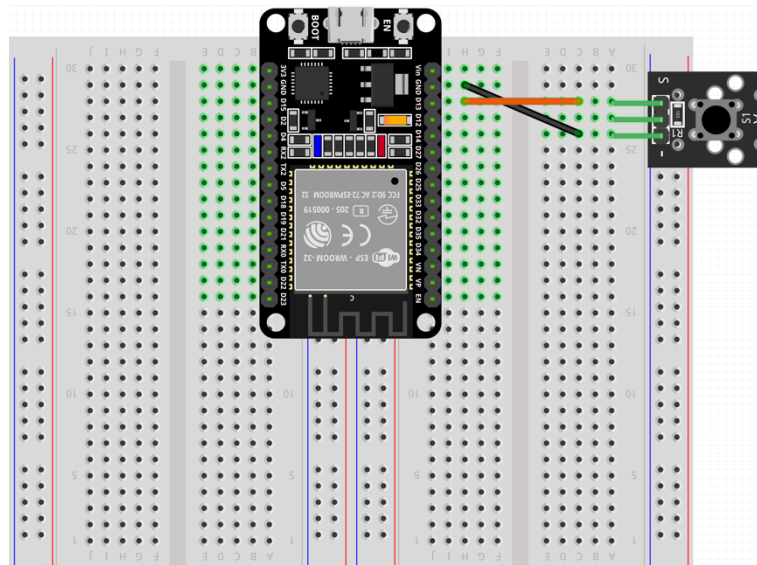
```
cmake_minimum_required(VERSION 3.16.0)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
idf_build_set_property(MODE ${MODE})
project(STR2024_P1_3)
```

Abre y modifica el segundo de la siguiente manera:

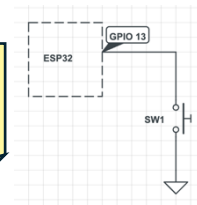
```
idf_build_get_property(MODE MODE)
if(MODE STREQUAL "polling")
    FILE(GLOB_RECURSE app_sources ${CMAKE_SOURCE_DIR}/src/polling/*.*)
elseif(MODE STREQUAL "interrupt")
    FILE(GLOB_RECURSE app_sources ${CMAKE_SOURCE_DIR}/src/interrupt/*.*)
else()
    message(FATAL_ERROR "Selecciona el modo polling o interrupt en Platformio.")
endif()
idf_component_register(SRCS ${app_sources})
```

Observa que Platformio.ini crea la variable MODE y la pasa a CMake, esta es visible en el CMakeLists.txt que está en la raíz del proyecto; sin embargo, el segundo CMakeList.txt no es invocado directamente por el primero, sino que se lanza desde los scripts propios de ESP-IDF (\$ENV{IDF_PATH}/tools/cmake/project.cmake). Por esta razón, para hacer esta variable visible en el CMakeLists.txt del directorio src, es necesario definir la variable MODE con idf_build_set_property en el primero, y acceder a la variable con idf_build_get_property en el segundo.

3. Monta en la protoboard el botón como aparece en la siguiente imagen.



La parte “visible” de nuestro montaje es un simple pulsador que interrumpe una conexión entre el pin (GPIO) y la masa (referencia 0V). Más adelante veremos que es un poco más complejo por “lo que no se ve”.



4. Crea el fichero “src/polling/main.c” e introduce el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

const char* TAG="P1_3";

void app_main(void)
{
    gpio_config_t io_conf = \
    {
        .intr_type    = GPIO_INTR_DISABLE,
        .mode         = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL<<CONFIG_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en   = GPIO_PULLUP_ENABLE
    };
    gpio_config(&io_conf);

    uint8_t gpio_value;
    while (1)
    {
        gpio_value = gpio_get_level(CONFIG_GPIO_PIN);
        if (gpio_value == 0)
```

```

{
    ESP_LOGI(TAG, "Botón pulsado!");
}
vTaskDelay(10 / portTICK_PERIOD_MS);
}
}

```

Y crea un Kconfig.projbuild en el directorio src con lo siguiente:

menu "Configuración Práctica 1 parte 3"

```

config GPIO_PIN
    int "GPIO Pin"
    range 1 39
    default 13
    help

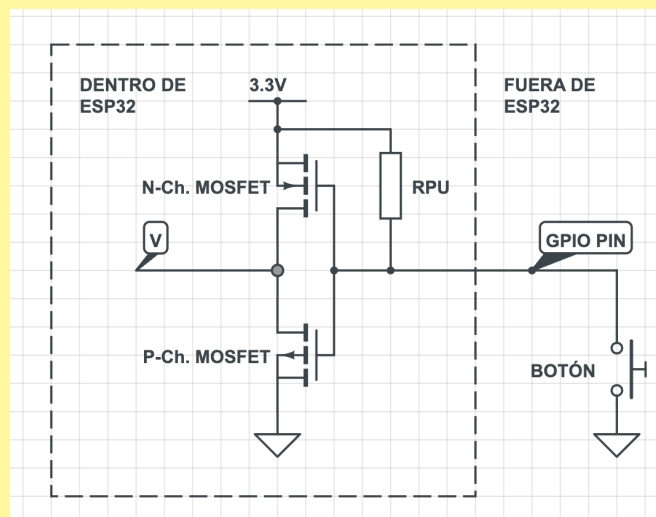
```

Número de puerto de entrada/salida de propósito general (GPIO) donde se conecta la señal del pulsador

endmenu

Analiza el código. ESP-IDF se ha diseñado con la misma filosofía para todos sus módulos: hay una estructura de configuración (que siempre se nombra como `X_config_t`) que, una vez configurada, se pasa como referencia a un método que la usa para hacerla efectiva.

Lo que se ha configurado es una entrada (`mode = GPIO_MODE_INPUT`), sin interrupción (`intr_type = GPIO_INTR_DISABLE`), que afecta al pin número `CONFIG_GPIO_PIN` (el que se haya configurado en Menuconfig) y con una resistencia de pull-up activa (`pull_up_en = GPIO_PULLUP_ENABLE`, `pull_down_en = GPIO_PULLDOWN_DISABLE`). Estas resistencias son altas (apenas dejan pasar corriente), porque sirven para establecer la señal en el caso de que ésta se encuentre en alta impedancia. Piensa que cuando el botón esta sin pulsar, el valor del cable externo no es 0 ni 3.3V, está sin conectar, "al aire", lo que llamamos alta impedancia (Z). Para que en esa situación se detecte un 1 lógico (3.3V) necesitamos habilitar una resistencia de PULL-UP, que ponga en el cable el voltaje deseado. Cuando el botón se pulsa, se une el cable con 0V, puesto que la resistencia de pull-up es alta, este voltaje no se va a ver afectado. Para entender esto, necesitas comprender este circuito, "lo que no se ve":



Las resistencias de Pull se pueden activar o desactivar por hardware según tu proyecto lo requiera.

Apartado V: Polling

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

1.- Compila y carga el firmware de la configuración polling desde Platformio. Utiliza la opción "Monitor" para visualizar los mensajes de la aplicación en ejecución. Pulsa el botón varias veces. Aporta capturas de pantalla.

2.- Como habrás observado, el código asociado al botón se ejecuta continuamente en bucle. Modifica el código para que únicamente se ejecute una vez por pulsación. Incluye tu código, su explicación, y capturas de pantalla en la memoria.

5. Crea el fichero "src/interrupt/main.c" e introduce el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "esp_err.h"

const char* TAG="P1_3";

// Manejador de interrupción (ISR)
static void IRAM_ATTR gpio_isr_handler(void* arg)
{
    ESP_LOGI(TAG, "Botón pulsado!");
}

void app_main(void)
{
    gpio_config_t io_conf = \
    {
        .intr_type    = GPIO_INTR_NEGEDGE,
        .mode         = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL<<CONFIG_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en   = GPIO_PULLUP_ENABLE
    };
    ESP_ERROR_CHECK(gpio_config(&io_conf));
    ESP_ERROR_CHECK(gpio_install_isr_service(ESP_INTR_FLAG_SHARED));
    ESP_ERROR_CHECK(gpio_isr_handler_add(CONFIG_GPIO_PIN, gpio_isr_handler,
    NULL));

    while (1)
    {
        vTaskDelay(10 / portTICK_PERIOD_MS);
    }
}
```

En esta ocasión hemos hecho nuestro código más seguro. Fíjate en la macro `ESP_ERROR_CHECK`, que está definida en "esp_err.h". Casi todas las funciones de la API de ESP-IDF devuelven códigos definidos con macros como `ESP_OK`, `ESP_FAIL` y otros. La macro `ESP_ERROR_CHECK` se asegura de que la invocación devuelve `ESP_OK` y, si no, fuerza el reinicio del sistema. Dedícale unos minutos a ver el fichero "esp_check.h"

Apartado VI: Interrupt

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

1.- Compila y carga el firmware de la configuración interrupts desde Platformio. Utiliza la opción "Monitor" para visualizar los mensajes de la aplicación en ejecución. Pulsa el botón. ¿Qué ha pasado?. Aporta capturas de pantalla.

2.- Dentro del código de la interrupción, sustituye la macro ESP_LOGI por ESP_DRAM_LOGI. Vuelve a compilar y cargar el firmware. ¿Por qué hemos tenido que hacer este cambio? Busca información en Internet.

Apartado VII: GPIO como salida

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

1.- En el pin GPIO 2 tienes un LED soldado en la placa. Modifica el código src/polling/main.c para que este led se apague cuando el botón se pulsa, y se encienda en caso contrario. Aporta código, explicación y capturas en la memoria.

2.- Haz lo mismo con interrupciones modificando el código src/interrupt/main.c. Aporta código, explicación y capturas en la memoria.